ETHANS CLOUD & AI ACADEMY

Operators in Python

Part 1

Arithmetic · Comparison · Assignment · Membership · Identity · Strings

Chapter 1

Arithmetic Operators

Performing Mathematical Calculations in Python

In every software application — from a simple calculator to a financial system — the program needs to perform mathematical operations. Python provides a dedicated set of symbols called arithmetic operators for this purpose. In this chapter, you will learn each operator in detail, understand how Python evaluates expressions, and practice with professional examples.

1.1 What is an Operator?

Operator	A special symbol or keyword that tells Python to perform a specific operation on one or more values. The values the operator works on are called operands.

In the expression `10 + 3`, the plus sign (+) is the operator, and 10 and 3 are the operands. The entire expression produces a result called the value of the expression.

🔎 Operator Terminology

Expression: `10 + 3`

```
      ↑ ↑ ↑
Operand Operator Operand
```

Result: 13

Python evaluates expressions and replaces them with their result whenever your program runs.

1.2 The Seven Arithmetic Operators

Python provides seven arithmetic operators. Each one is listed below with its symbol, name, and purpose.

Operator	Name	Description	Example	Result
+	Addition	Adds two numbers together	15 + 4	19
-	Subtraction	Subtracts the right operand from the left	15 - 4	11
*	Multiplication	Multiplies two numbers	15 * 4	60
/	Division	Divides left by right — result is always a float	15 / 4	3.75
//	Floor Division	Divides and returns only the whole number (integer) part	15 // 4	3
%	Modulus	Returns the remainder after division	15 % 4	3
**	Exponentiation	Raises the left number to the power of the right	2 ** 8	256

Addition (+)

The addition operator adds two numeric values. It is the most straightforward arithmetic operator.

```
# Addition examples:
basic_salary    = 45000
hra             = 12000      # House Rent Allowance
transport       = 3500

gross_salary    = basic_salary + hra + transport
print('Gross Salary:', gross_salary)
# Output: Gross Salary: 60500

# Adding floats:
product_price   = 1299.00
gst_amount      = 233.82     # 18% GST
total_price     = product_price + gst_amount
print('Total with GST:', total_price)
# Output: Total with GST: 1532.82
```

Figure 1.1 — Addition operator

▸ **Output**

```
Gross Salary: 60500

Total with GST: 1532.82
```

Subtraction (-)

The subtraction operator deducts the right operand from the left operand.

```
# Subtraction examples:
total_budget    = 100000
amount_spent    = 67450
remaining       = total_budget - amount_spent
print('Remaining Budget:', remaining)
# Output: Remaining Budget: 32550

# Subtraction with floats:
full_marks      = 100.0
marks_deducted  = 8.5
final_score     = full_marks - marks_deducted
print('Final Score:', final_score)
# Output: Final Score: 91.5
```

Figure 1.2 — Subtraction operator

▸ **Output**

```
Remaining Budget: 32550

Final Score: 91.5
```

Multiplication (*)

The multiplication operator multiplies two values together.

```
# Multiplication examples:
unit_price      = 250
quantity        = 48
total_cost      = unit_price * quantity
print('Total Cost:', total_cost)
# Output: Total Cost: 12000

# Compound interest calculation:
principal       = 50000
rate            = 0.08        # 8% annual interest rate
years           = 3
interest        = principal * rate * years
print('Total Interest:', interest)
# Output: Total Interest: 12000.0
```

Figure 1.3 — Multiplication operator

▸ **Output**

```
Total Cost: 12000

Total Interest: 12000.0
```

Division (/)

The division operator divides the left operand by the right. In Python 3, the result is always a float — even when both numbers are whole numbers and divide evenly.

```
# Division examples:
total_marks     = 450
subjects        = 5
average         = total_marks / subjects
print('Average Marks:', average)
# Output: Average Marks: 90.0    <-- Note: result is a float

# Division that results in a decimal:
total_distance  = 285
fuel_used       = 12
mileage         = total_distance / fuel_used
print('Mileage (km per litre):', mileage)
# Output: Mileage (km per litre): 23.75
```

Figure 1.4 — Division operator

▸ **Output**

```
Average Marks: 90.0

Mileage (km per litre): 23.75
```

📝 **Note**

In Python 3, dividing two integers with / always produces a float. So 10 / 2 gives 5.0, not 5. If you need an integer result, use floor division (//) instead.

Floor Division (//)

Floor division divides two numbers and discards (floors) the decimal part, returning only the whole number portion of the result.

```
# Floor division examples:
total_items     = 97
items_per_box   = 12
full_boxes      = total_items // items_per_box
print('Complete Boxes:', full_boxes)
# Output: Complete Boxes: 8
```

```
# Time conversion — minutes to hours:
total_minutes   = 250
hours           = total_minutes // 60
print('Complete Hours:', hours)
# Output: Complete Hours: 4

# Comparing / and //:
print(17 / 5)     # Output: 3.4   (regular division)
print(17 // 5)    # Output: 3     (floor division — decimal dropped)
```

Figure 1.5 — Floor division operator

▸ Output

Complete Boxes: 8

Complete Hours: 4

3.4

3

Modulus (%)

The modulus operator returns the remainder that is left over after division. It is extremely useful for tasks like checking whether a number is even or odd, or converting units.

```
# Modulus examples:
total_items     = 97
items_per_box   = 12
leftover        = total_items % items_per_box
print('Items Left Over:', leftover)
# Output: Items Left Over: 1

# Minutes remaining after extracting full hours:
total_minutes   = 250
remaining_mins  = total_minutes % 60
print('Minutes Remaining:', remaining_mins)
# Output: Minutes Remaining: 10

# Checking if a number is even or odd:
number          = 47
if number % 2 == 0:
    print(number, 'is Even')
else:
    print(number, 'is Odd')
# Output: 47 is Odd
```

Figure 1.6 — Modulus operator

▸ **Output**

Items Left Over: 1

Minutes Remaining: 10

47 is Odd

🔎 **Floor Division and Modulus Together**

Floor division and modulus are often used together to break a total into units.

Example: Convert 250 minutes into hours and minutes.

total_minutes = 250

hours = 250 // 60 --> 4

minutes = 250 % 60 --> 10

Result: 4 hours and 10 minutes.

Exponentiation (**)

The exponentiation operator raises a number (the base) to a given power (the exponent). It is the Python equivalent of writing x^n in mathematics.

```
# Exponentiation examples:
# 2 to the power of 10:
result = 2 ** 10
print('2^10 =', result)
# Output: 2^10 = 1024

# Area of a square:
side   = 15
area   = side ** 2
print('Area of square:', area, 'sq. units')
# Output: Area of square: 225 sq. units

# Volume of a cube:
edge   = 4
volume = edge ** 3
print('Volume of cube:', volume, 'cubic units')
# Output: Volume of cube: 64 cubic units

# Square root using exponentiation (power of 0.5):
```

```
number = 144
sqrt   = number ** 0.5
print('Square root of 144:', sqrt)
# Output: Square root of 144: 12.0
```

Figure 1.7 — Exponentiation operator

▸ **Output**

2^10 = 1024

Area of square: 225 sq. units

Volume of cube: 64 cubic units

Square root of 144: 12.0

1.3 Order of Operations (Operator Precedence)

When an expression contains multiple operators, Python does not evaluate them left to right. It follows a defined order called operator precedence — the same rules you learned in school mathematics (BODMAS / PEMDAS).

Priority	Operator(s)	Operation	Evaluated...
1 (Highest)	**	Exponentiation	Right to left
2	+ -	Unary plus, unary minus	Right to left
3	* / // %	Multiplication, Division, Floor, Modulus	Left to right
4 (Lowest)	+ -	Addition, Subtraction	Left to right

```
# Precedence examples:
result = 2 + 3 * 4
print(result)        # Output: 14   (not 20 — multiplication first)

result = (2 + 3) * 4
print(result)        # Output: 20   (parentheses override — evaluated first)

result = 2 ** 3 + 4 * 2
print(result)        # Output: 16   (8 + 8 — exponent first, then multiply,
then add)

# Real example — EMI calculation (simplified):
principal   = 500000
rate        = 0.01      # 1% monthly interest
months      = 24
```

```
emi         = principal * rate * (1 + rate) ** months / ((1 + rate) **
months - 1)
print('Monthly EMI: Rs.', round(emi, 2))
```

Figure 1.8 — Operator precedence examples

▸ Output

14

20

16

Monthly EMI: Rs. 23536.75

💡 Pro Tip

When in doubt about operator precedence, use parentheses ( ) to explicitly control the order of evaluation. This also makes your code easier to read and understand.

Chapter 2

Comparison Operators

Making Decisions by Comparing Values

Almost every real-world program needs to make decisions — should the user be allowed to log in? Has the stock level fallen below the minimum? Did the student pass the exam? These decisions require comparing values. Python provides comparison operators specifically for this purpose.

Comparison Operator An operator that compares two values and returns a Boolean result — either True or False. These are also called relational operators.

2.1 The Six Comparison Operators

Operator	Meaning	Returns True When...	Example	Result
==	Equal to	Both values are identical	10 == 10	True
!=	Not equal to	Values are different	10 != 5	True
>	Greater than	Left value is larger	15 > 10	True
<	Less than	Left value is smaller	7 < 12	True
>=	Greater than or equal	Left is larger than or equal to right	10 >= 10	True
<=	Less than or equal	Left is smaller than or equal to right	8 <= 10	True

⚠ **Watch Out**

Do not confuse = (assignment) with == (comparison). Writing if age = 18 will cause a SyntaxError. The single equals sign stores a value; the double equals sign compares two values.

Equal to (==) and Not Equal to (!=)

The == operator checks whether two values are identical. The != operator checks whether two values are different. Both return a Boolean result.

```
# Equal to and Not Equal to:
entered_pin     = 4821
correct_pin     = 4821

print(entered_pin == correct_pin)    # Output: True
print(entered_pin != correct_pin)    # Output: False

# String comparison:
user_role       = 'admin'
required_role   = 'manager'

print(user_role == required_role)    # Output: False
print(user_role != required_role)    # Output: True
```

```
# Practical use — login verification:
username        = 'alice'
password        = 'secure123'

if username == 'alice' and password == 'secure123':
    print('Login Successful. Welcome, Alice.')
else:
    print('Invalid credentials. Please try again.')
```

Figure 2.1 — Equal to and Not Equal to operators

▸ Output

```
True
False
False
True
Login Successful. Welcome, Alice.
```

Greater Than (>) and Less Than (<)

These operators compare the magnitude of two values. They are used extensively in conditional checks and business logic.

```
# Grading system based on marks:
marks           = 78

print(marks > 90)     # Output: False
print(marks > 70)     # Output: True
print(marks < 50)     # Output: False

# Inventory alert:
current_stock   = 15
minimum_stock   = 20

if current_stock < minimum_stock:
    print('Alert: Stock is below minimum level. Please reorder.')
else:
    print('Stock level is adequate.')

# Temperature check:
cpu_temp        = 92
safe_limit      = 85
```

```
if cpu_temp > safe_limit:
    print('Warning: CPU temperature is above safe limit!')
```

Figure 2.2 — Greater than and Less than operators

▸ **Output**

False

True

False

Alert: Stock is below minimum level. Please reorder.

Warning: CPU temperature is above safe limit!

Greater Than or Equal To (>=) and Less Than or Equal To (<=)

These operators check whether a value is within a range boundary — including the boundary value itself.

```
# Eligibility check — age 18 or above:
applicant_age   = 18

if applicant_age >= 18:
    print('Eligible to apply.')
else:
    print('Minimum age requirement not met.')
# Output: Eligible to apply.

# Exam passing threshold:
pass_mark       = 40
student_score   = 40

if student_score >= pass_mark:
    print('Result: PASS')
else:
    print('Result: FAIL')
# Output: Result: PASS

# Data validation — discount only for orders up to 1000:
order_value     = 850
if order_value <= 1000:
    discount = order_value * 0.05
    print('Discount Applied:', discount)
# Output: Discount Applied: 42.5
```

Figure 2.3 — Greater/Less than or equal to operators

▸ **Output**

Eligible to apply.

Result: PASS

Discount Applied: 42.5

2.2 Chaining Comparison Operators

Python allows you to chain multiple comparison operators together in a single expression, making range checks very clean and readable — much closer to how they are written in mathematics.

```
# Chained comparisons — checking ranges:
temperature     = 24

# Is temperature in the comfortable range (18 to 30)?
if 18 <= temperature <= 30:
    print('Temperature is comfortable.')
# Output: Temperature is comfortable.

# Grading with chained comparisons:
score           = 78

if score >= 90:
    grade = 'A'
elif 75 <= score < 90:
    grade = 'B'
elif 60 <= score < 75:
    grade = 'C'
elif 40 <= score < 60:
    grade = 'D'
else:
    grade = 'F'

print('Grade:', grade)
# Output: Grade: B
```

Figure 2.4 — Chained comparison operators

▸ **Output**

Temperature is comfortable.

Grade: B

Chapter 3

Assignment Operators

Storing and Updating Values in Variables

You have already used the basic assignment operator (=) to store values in variables. Python provides a complete family of assignment operators that combine assignment with arithmetic — making common update operations shorter and more readable.

Assignment Operator — An operator used to assign a value to a variable. Compound assignment operators perform an arithmetic operation and then assign the result back to the same variable in one step.

3.1 All Assignment Operators

Operator	Full Form Equivalent	Meaning	Example	Result (if x = 10)
=	x = value	Simple assignment — stores a value	x = 10	x is 10
+=	x = x + value	Add and assign	x += 5	x is 15
-=	x = x - value	Subtract and assign	x -= 3	x is 7
*=	x = x * value	Multiply and assign	x *= 2	x is 20
/=	x = x / value	Divide and assign (result is float)	x /= 4	x is 2.5
//=	x = x // value	Floor divide and assign	x //= 3	x is 3
%=	x = x % value	Modulus and assign (store remainder)	x %= 3	x is 1
**=	x = x ** value	Exponentiate and assign	x **= 2	x is 100

Simple Assignment (=)

The basic assignment operator stores a value into a variable. This is the most fundamental operation in programming.

```
# Simple assignment:
employee_name   = 'Rahul Sharma'
department      = 'Technology'
annual_ctc      = 1200000
joining_date    = '2022-06-01'
```

```
print(employee_name, '|', department, '|', annual_ctc)
# Output: Rahul Sharma | Technology | 1200000
```

Figure 3.1 — Simple assignment operator

▸ **Output**

```
Rahul Sharma | Technology | 1200000
```

Compound Assignment Operators — Building a Running Total

Compound assignment operators are particularly useful when you need to update a variable repeatedly — such as adding items to a cart, accumulating totals, or counting occurrences.

```
# Building a shopping cart total using compound assignment:
cart_total  = 0

# Adding items one by one:
cart_total  += 1299      # Keyboard
print('After Keyboard:', cart_total)

cart_total  += 2499      # Mouse
print('After Mouse:', cart_total)

cart_total  += 8999      # Headphones
print('After Headphones:', cart_total)

# Apply 10% discount:
cart_total  *= 0.9
print('After 10% Discount:', cart_total)

# Add delivery charge:
cart_total  += 149
print('Final Amount to Pay:', cart_total)
```

Figure 3.2 — Compound assignment operators in a shopping cart

▸ **Output**

```
After Keyboard: 1299
After Mouse: 3798
After Headphones: 12797
After 10% Discount: 11517.3
Final Amount to Pay: 11666.3
```

Practical Use — Counter and Accumulator Patterns

Two of the most common patterns in programming are the counter (increment a number by 1 each time an event occurs) and the accumulator (keep a running total). Compound assignment operators are ideal for both.

```
# Counter pattern — count failed login attempts:
failed_attempts = 0

failed_attempts += 1    # First wrong password
failed_attempts += 1    # Second wrong password
failed_attempts += 1    # Third wrong password

print('Failed Attempts:', failed_attempts)

if failed_attempts >= 3:
    print('Account temporarily locked for security.')

# Accumulator pattern — total sales across 4 quarters:
annual_sales    = 0
annual_sales   += 425000    # Q1
annual_sales   += 510000    # Q2
annual_sales   += 390000    # Q3
annual_sales   += 620000    # Q4

print('Annual Sales:', annual_sales)
# Output: Annual Sales: 1945000
```

Figure 3.3 — Counter and accumulator patterns

▸ Output

Failed Attempts: 3

Account temporarily locked for security.

Annual Sales: 1945000

🧪 Try It Yourself

Open Python IDLE or VS Code.

Create a variable balance = 5000.

Use compound assignment to:

— Add a credit of 2500

— Deduct an expense of 800

— Apply a 2% monthly interest (multiply by 1.02)

Print the final balance after each step.

Chapter 4

Membership and Identity Operators

Checking Belonging and Object Identity

Beyond comparing values mathematically, Python provides two specialized types of operators: membership operators that test whether a value exists within a collection, and identity operators that check whether two variables point to the exact same object in memory.

4.1 Membership Operators — `in` and `not in`

Membership Operator An operator that tests whether a value is present within a sequence or collection (such as a string, list, tuple, or set). Returns True or False.

Operator	Meaning	Returns True When...
`in`	Membership test	The value exists in the sequence or collection
`not in`	Negative membership test	The value does NOT exist in the sequence or collection

Using `in` with Lists

The in operator is frequently used with lists to check whether a particular item is present before performing an action on it.

```
# Membership in a list:
approved_vendors    = ['TechSupply Co.', 'CloudNine Ltd.', 'DataPro Inc.']

vendor              = 'CloudNine Ltd.'
print(vendor in approved_vendors)
# Output: True

vendor              = 'ShadyDeals Corp.'
print(vendor in approved_vendors)
# Output: False

# Practical — only allow approved vendors:
new_vendor          = 'TechSupply Co.'
if new_vendor in approved_vendors:
    print('Vendor verified. Purchase order approved.')
else:
    print('Vendor not on approved list. Request blocked.')
```

Figure 4.1 — Membership operator with lists

▶ **Output**

```
True
False
Vendor verified. Purchase order approved.
```

Using `in` with Strings

When used with a string, the in operator checks whether one string is a substring of another — i.e., whether it appears anywhere within the larger string.

```
# Membership in strings:
email_address   = 'alice.sharma@company.com'

# Check if it is a company email:
print('@company.com' in email_address)
# Output: True

# Check if it is a Gmail address:
print('@gmail.com' in email_address)
# Output: False

# Content moderation — check for restricted words:
user_message    = 'Please review the attached invoice and approve.'
keywords        = ['invoice', 'payment', 'transfer']

for word in keywords:
    if word in user_message:
        print(f'Keyword found: {word}')
# Output: Keyword found: invoice
```

Figure 4.2 — Membership operator with strings

▶ **Output**

```
True
False
Keyword found: invoice
```

Using `not in` for Exclusion Checks

The not in operator is useful when you need to verify that something is absent — for example, that a username is not already taken, or that an item is not in a blocked list.

```
# not in examples:
registered_users = ['alice', 'bob', 'charlie', 'diana']
```

```
new_user         = 'edward'
if new_user not in registered_users:
    print(f'Username {new_user!r} is available. Registration successful.')
else:
    print(f'Username {new_user!r} is already taken.')
# Output: Username 'edward' is available. Registration successful.

# Blocked IP addresses:
blocked_ips      = ['192.168.1.5', '10.0.0.22', '172.16.0.3']
request_ip       = '192.168.1.10'

if request_ip not in blocked_ips:
    print('Access granted for IP:', request_ip)
else:
    print('Access denied. IP is blocked.')
# Output: Access granted for IP: 192.168.1.10
```

Figure 4.3 — not in operator

▸ **Output**

Username 'edward' is available. Registration successful.

Access granted for IP: 192.168.1.10

4.2 Identity Operators — is and is not

Identity Operator An operator that checks whether two variables refer to the exact same object in memory (not just whether they have equal values). Returns True or False.

Operator	Meaning	Returns True When...
`is`	Identity test	Both variables point to the same object in memory
`is not`	Negative identity test	The two variables point to different objects in memory

🔎 Value Equality (==) vs. Identity (is) — The Key Difference

== asks: Do these two variables hold the same VALUE?

is asks: Are these two variables the SAME OBJECT in memory?

Two variables can hold equal values but still be different objects:

list_a = [1, 2, 3]

list_b = [1, 2, 3]

list_a == list_b --> True (same values)

list_a is list_b --> False (different objects in memory)

```
# Identity vs. equality:
list_a = [10, 20, 30]
list_b = [10, 20, 30]
list_c = list_a          # list_c points to the same object as list_a

print(list_a == list_b)   # True  — same values
print(list_a is  list_b)  # False — different objects
print(list_a is  list_c)  # True  — same object

# The most common use of 'is' — checking for None:
query_result    = None

if query_result is None:
    print('No data returned from the database.')
else:
    print('Records found:', query_result)
# Output: No data returned from the database.

# Using 'is not' — confirming a result exists:
user_input      = 'Confirm'
if user_input is not None:
    print('Processing input:', user_input)
# Output: Processing input: Confirm
```

Figure 4.4 — Identity operators is and is not

▸ Output

True

False

True

No data returned from the database.

Processing input: Confirm

📝 Note

The primary use case for the is operator in professional Python code is checking for None. Always write if variable is None rather than if variable == None. This is the recommended Python style (PEP 8).

Chapter 5

Data Handling — Integers and Strings

Converting, Processing, and Working with int and str

In real-world programs, data does not always arrive in the format you need. A user types a number but Python receives it as text. You need to display a calculation result alongside a label. This chapter covers how Python handles conversions between integers and strings, and the essential operations you will use with both types.

5.1 Type Conversion — int and str

Type Conversion	The process of changing a value from one data type to another. Also called type casting. Python provides built-in functions to convert between int, float, and str.

Converting String to Integer — int()

When a user enters a number through a program (e.g., using the input() function), Python stores it as a string by default. To perform arithmetic on it, you must first convert it to an integer using the int() function.

```
# The input() function always returns a string:
age_input   = input('Enter your age: ')     # Suppose user types: 25
print(type(age_input))                      # Output: <class 'str'>

# Converting to integer:
age         = int(age_input)
print(type(age))                            # Output: <class 'int'>

# Now arithmetic is possible:
retirement_age = 60
years_left     = retirement_age - age
print('Years until retirement:', years_left)
# Output: Years until retirement: 35

# Direct conversion in one line:
salary      = int(input('Enter monthly salary: '))   # Converts immediately
annual_ctc  = salary * 12
print('Annual CTC:', annual_ctc)
```

Figure 5.1 — Converting string to integer

⚠ Watch Out

int() will fail if the string contains non-numeric characters.

int("25") --> Works fine — result: 25

int("25.5") --> ValueError — float string cannot convert directly to int

int("twenty") --> ValueError — text cannot convert to int

To convert a decimal string to integer, first convert to float: int(float('25.5')) --> 25

Converting Integer to String — str()

When you want to combine a number with text (for example, in a message or report), you must convert the number to a string using str(). This is called string concatenation with type conversion.

```
# Why conversion is needed:
order_id    = 10042
# print('Order ID: ' + order_id)     # This causes a TypeError!

# Correct approach — convert number to string:
print('Order ID: ' + str(order_id))
# Output: Order ID: 10042

# Building a formatted report line:
employee_id = 1021
name        = 'Priya Mehta'
salary      = 75000

report_line = 'EMP-' + str(employee_id) + ' | ' + name + ' | Rs.' +
str(salary)
print(report_line)
# Output: EMP-1021 | Priya Mehta | Rs.75000

# Preferred modern approach — f-strings:
report_line = f'EMP-{employee_id} | {name} | Rs.{salary}'
print(report_line)
# Output: EMP-1021 | Priya Mehta | Rs.75000
```

Figure 5.2 — Converting integer to string

▸ **Output**

Order ID: 10042

EMP-1021 | Priya Mehta | Rs.75000

Pro Tip

Use f-strings (f'...') for combining variables and text — they are the modern, preferred way in Python 3. Simply prefix the string with f and write variable names inside curly braces { }. No type conversion needed!

5.2 Essential Integer Operations

Built-in Functions for Integers

Python provides several built-in functions that are commonly used with integers:

Function	Purpose	Example	Result
abs()	Returns the absolute (positive) value	abs(-42)	42
round()	Rounds a float to the nearest integer	round(3.7)	4
pow()	Raises a number to a power (same as **)	pow(2, 10)	1024
max()	Returns the largest value among arguments	max(8, 3, 15)	15
min()	Returns the smallest value among arguments	min(8, 3, 15)	3
sum()	Returns the sum of items in an iterable	sum([4,8,12])	24

```
# Practical use of integer functions:
temperatures    = [32, -5, 28, -12, 19, 41]

print('Highest temperature:', max(temperatures))
print('Lowest temperature:', min(temperatures))
print('Total sum:', sum(temperatures))
print('Average:', sum(temperatures) / len(temperatures))

# abs() for absolute difference:
budget          = 50000
actual_spend    = 53750
variance        = abs(budget - actual_spend)
print('Budget Variance: Rs.', variance)

# round() for currency formatting:
total_with_gst  = 1845.678
print('Rounded amount:', round(total_with_gst, 2))
# Output: Rounded amount: 1845.68
```

Figure 5.3 — Integer utility functions

▸ **Output**

```
Highest temperature: 41
Lowest temperature: -12
Total sum: 103
Average: 17.166666666666668
Budget Variance: Rs. 3750
Rounded amount: 1845.68
```

5.3 Essential String Operations

Strings are among the most frequently used data types in any program. Python provides a rich set of methods to search, modify, clean, and format string data.

Case Conversion Methods

```
name    = 'alice SHARMA'

print(name.upper())       # Output: ALICE SHARMA
print(name.lower())       # Output: alice sharma
print(name.title())       # Output: Alice Sharma
print(name.capitalize())  # Output: Alice sharma  (only first letter
capitalised)

# Practical — normalising user input for comparison:
user_input  = '  YES  '
if user_input.strip().upper() == 'YES':
    print('User confirmed.')
```

Figure 5.4 — String case conversion methods

▸ **Output**

```
ALICE SHARMA
alice sharma
Alice Sharma
Alice sharma
User confirmed.
```

Searching and Checking String Content

```
product_code    = 'PRD-2025-LAPTOP-001'
```

```
# Check if a substring exists:
print('LAPTOP' in product_code)             # Output: True
print(product_code.startswith('PRD'))        # Output: True
print(product_code.endswith('999'))          # Output: False

# Find position of a substring:
print(product_code.find('LAPTOP'))           # Output: 9
print(product_code.find('PHONE'))            # Output: -1  (not found)

# Count occurrences:
sentence = 'data science uses data from data sources'
print(sentence.count('data'))                # Output: 3

# Validation checks:
pin_code    = '380015'
print(pin_code.isdigit())                    # Output: True

username    = 'alice_sharma'
print(username.isalpha())                    # Output: False (has
underscore)
print(username.isalnum())                    # Output: False (has
underscore)
```

Figure 5.5 — String search and check methods

▸ **Output**

```
True
True
False
9
-1
3
True
False
False
```

Cleaning and Modifying Strings

```
# strip() — remove leading and trailing whitespace:
user_email  = '   alice@example.com   '
clean_email = user_email.strip()
print(repr(clean_email))
# Output: 'alice@example.com'
```

```
# replace() — substitute one substring with another:
message     = 'Dear Customer, your order ORD001 is confirmed.'
updated     = message.replace('Customer', 'Alice').replace('ORD001',
'ORD-2025-001')
print(updated)
# Output: Dear Alice, your order ORD-2025-001 is confirmed.

# split() — divide a string into a list:
csv_row     = 'Alice,Engineering,Mumbai,95000'
fields      = csv_row.split(',')
print(fields)
# Output: ['Alice', 'Engineering', 'Mumbai', '95000']
print('Name:', fields[0])
print('City:', fields[2])

# join() — combine a list of strings into one:
words       = ['Python', 'is', 'a', 'powerful', 'language']
sentence    = ' '.join(words)
print(sentence)
# Output: Python is a powerful language
```

Figure 5.6 — String cleaning and modification methods

▸ **Output**

'alice@example.com'

Dear Alice, your order ORD-2025-001 is confirmed.

['Alice', 'Engineering', 'Mumbai', '95000']

Name: Alice

City: Mumbai

Python is a powerful language

Chapter 6

Indexing and Slicing in Strings

Accessing and Extracting Parts of a String

One of the most powerful features of strings in Python is the ability to access any character or group of characters by their position. This is called indexing and slicing. Mastering these two techniques allows you to extract, process, and transform text data precisely — a skill used constantly in data processing, file handling, and text analysis.

6.1 String Indexing

String Index	A numeric position that identifies the location of a character within a string. Python indexes start at 0 for the first character. Negative indexes count from the end of the string.

Consider the string: 'PYTHON'

P	Y	T	H	O	N
0	1	2	3	4	5
-6	-5	-4	-3	-2	-1

Figure 6.1 — Positive (blue) and negative (orange) indexes for the string 'PYTHON'

Positive indexes (shown in blue) count from left to right, starting at 0.

Negative indexes (shown in orange) count from right to left, starting at -1 for the last character.

```
language    = 'PYTHON'

# Positive indexing:
print(language[0])     # Output: P   (first character)
print(language[1])     # Output: Y
print(language[3])     # Output: H
print(language[5])     # Output: N   (last character, index = length - 1)

# Negative indexing:
print(language[-1])    # Output: N   (last character)
print(language[-2])    # Output: O   (second from last)
print(language[-6])    # Output: P   (same as index 0)
```

```
# Finding string length:
print(len(language))    # Output: 6
```

Figure 6.2 — Positive and negative string indexing

▸ **Output**

```
P
Y
H
N
N
O
P
6
```

🔎 **Negative Indexing — When Is It Useful?**

Negative indexes are especially useful when you need the last few characters of a string, regardless of how long the string is.

Example: Extract the file extension from a filename.

```
filename = "annual_report_2025.xlsx"
last_4   = filename[-4:]    -->  'xlsx'
```

No matter how long the filename is, the last 4 characters are always the extension.

6.2 String Slicing

Slicing	Extracting a portion (substring) of a string by specifying a start index, an end index, and an optional step value. The syntax is: string[start : end : step]

Slicing follows three rules you must remember:

1. **Start index** — The position where the slice begins (inclusive).
2. **End index** — The position where the slice stops (exclusive — the character at this position is NOT included).
3. **Step** — How many positions to jump between each character (default is 1).

Syntax	Meaning
string[start:end]	Extract from index start up to (not including) end
string[start:]	Extract from index start to the end of the string
string[:end]	Extract from the beginning up to (not including) end
string[:]	Extract the entire string (full copy)
string[start:end:step]	Extract every step-th character from start up to end
string[::-1]	Reverse the entire string

Basic Slicing

```
product_code    = 'PRD-2025-LAPTOP-001'

# Extract different parts:
prefix          = product_code[0:3]
print('Prefix:', prefix)             # Output: Prefix: PRD

year            = product_code[4:8]
print('Year:', year)                 # Output: Year: 2025

category        = product_code[9:15]
print('Category:', category)         # Output: Category: LAPTOP

serial          = product_code[16:]
print('Serial:', serial)             # Output: Serial: 001

# Slicing from the beginning:
first_five      = product_code[:5]
print('First 5:', first_five)        # Output: First 5: PRD-2
```

Figure 6.3 — Basic string slicing

▸ **Output**

```
Prefix: PRD
Year: 2025
Category: LAPTOP
Serial: 001
First 5: PRD-2
```

Slicing with Negative Indexes

```
filename    = 'financial_report_Q4_2025.xlsx'

# Extract file extension (last 4 characters):
extension   = filename[-4:]
print('Extension:', extension)
# Output: Extension: xlsx

# Extract last 7 characters:
print(filename[-7:])
# Output: 25.xlsx

# Remove the last 5 characters (extension + dot):
name_only   = filename[:-5]
print('Name only:', name_only)
# Output: Name only: financial_report_Q4_2025

# Slice from a negative start to a negative end:
print(filename[-11:-5])
# Output: Q4_202
```

Figure 6.4 — Slicing with negative indexes

▸ **Output**

Extension: xlsx

25.xlsx

Name only: financial_report_Q4_2025

Q4_202

Slicing with Step

The step parameter controls how many positions Python jumps between each character it picks. The default step is 1 (every character). A step of 2 picks every second character; a step of -1 reverses the string.

```
text    = 'ABCDEFGHIJ'

# Every second character:
print(text[::2])          # Output: ACEGI

# Every third character:
print(text[::3])          # Output: ADGJ

# Every second character from index 1:
print(text[1::2])         # Output: BDFHJ

# Reverse the entire string:
print(text[::-1])         # Output: JIHGFEDCBA
```

```
# Reverse a portion (index 0 to 6, reversed):
print(text[6::-1])        # Output: GFEDCBA

# Practical — check if a word is a palindrome:
word    = 'racecar'
if word == word[::-1]:
    print(word, 'is a palindrome.')
# Output: racecar is a palindrome.
```

Figure 6.5 — Slicing with step parameter

▸ **Output**

```
ACEGI
ADG J
BDFHJ
JIHGFEDCBA
GFEDCBA
racecar is a palindrome.
```

6.3 Practical Slicing Examples

The following examples demonstrate how indexing and slicing are used in realistic scenarios.

Extracting Information from Structured Strings

```
# Employee ID format: DIV-YEAR-SERIAL  e.g. ENG-2023-0042
employee_id     = 'ENG-2023-0042'

division        = employee_id[:3]
joining_year    = employee_id[4:8]
serial_number   = employee_id[9:]

print('Division    :', division)
print('Joining Year:', joining_year)
print('Serial No.  :', serial_number)
```

Figure 6.6 — Extracting structured string data

▸ **Output**

```
Division    : ENG
Joining Year: 2023
Serial No.  : 0042
```

Masking Sensitive Information

```
# Mask a credit card number — show only last 4 digits:
card_number     = '4539578763621486'
last_four       = card_number[-4:]
masked          = '*' * 12 + last_four
print('Card Number (masked):', masked)
# Output: Card Number (masked): ************1486

# Mask email — show only first 3 chars and domain:
email           = 'alice.sharma@company.com'
at_pos          = email.find('@')
masked_email    = email[:3] + '***' + email[at_pos:]
print('Email (masked):', masked_email)
# Output: Email (masked): ali***@company.com
```

Figure 6.7 — Masking sensitive data with slicing

▸ Output

Card Number (masked): ************1486

Email (masked): ali***@company.com

Reversing and Validating Strings

```
# Reverse a string to verify a token:
original_token  = 'TKN20250001'
received_token  = '1000520NKT'

if original_token == received_token[::-1]:
    print('Token verified — reversed match confirmed.')
else:
    print('Token verification failed.')
# Output: Token verified — reversed match confirmed.

# Extract domain from email address:
email           = 'rahul.verma@techcorp.in'
at_index        = email.find('@')
domain          = email[at_index + 1:]
print('Domain:', domain)
# Output: Domain: techcorp.in
```

Figure 6.8 — Reversing and domain extraction

▸ Output

Token verified — reversed match confirmed.

Domain: techcorp.in

.4 Indexing and Slicing — Quick Reference

Use this reference table whenever you need a quick reminder of slicing syntax.

Task	Syntax	Example (s = 'PROGRAMMING')	Result
Get first character	s[0]	s[0]	P
Get last character	s[-1]	s[-1]	G
Get character at position n	s[n]	s[4]	R
Slice from start to end-1	s[a:b]	s[0:4]	PROG
Slice from index a to end	s[a:]	s[7:]	MING
Slice from start to index b	s[:b]	s[:4]	PROG
Copy entire string	s[:]	s[:]	PROGRAMMING
Every 2nd character	s[::2]	s[::2]	PORMIG
Reverse string	s[::-1]	s[::-1]	GNIMMARGORP
Last n characters	s[-n:]	s[-4:]	MING
All except last n	s[:-n]	s[:-4]	PROGRAM

📷 **SCREENSHOT PLACEHOLDER**

Screenshot: Python IDLE — showing an interactive session where the slicing examples from the Quick Reference table above are run one by one, each with its output

Chapter Summary

This module has covered the complete set of operators used in Python, along with practical data handling techniques for integers and strings. Here is a concise recap:

- **Arithmetic Operators** (+, -, *, /, //, %, **) perform mathematical calculations. Operator precedence (BODMAS) determines evaluation order.
- **Comparison Operators** (==, !=, >, <, >=, <=) compare two values and return True or False. Used in conditions and decision-making.
- **Assignment Operators** (=, +=, -=, *=, etc.) store or update variable values. Compound operators combine arithmetic and assignment in one step.
- **Membership Operators** (in, not in) check whether a value exists within a collection or string.
- **Identity Operators** (is, is not) check whether two variables point to the same memory object. Primarily used to test for None.
- **Type Conversion** between int and str is done using int() and str(). f-strings provide a clean way to embed values in text.

- **String Indexing** uses zero-based positive indexes from the left, and negative indexes from the right.
- **String Slicing** using string[start:end:step] extracts substrings with precise control over position and character selection.
